

THE NORTHCAP UNIVERSITY



PROJECT REPORT

SMART SAFETY SYSTEM FOR TWO WHEELERS

Submitted By: Chehak Arora (25CSU054)

Deepika (25CSU061)

Divyam Jain (25CSU068)

Course: B.TECH(CSE)

Semester: IInd

Section: B

Subject: Basics of Electrical and Electronics Engineering

Subject Code: ECL 110

Department: School of Engineering and Technology

Submitted To: Ms. Anjali Garg

INTRODUCTION

This project focuses on improving the safety of two-wheeler riders by integrating multiple sensors with an ESP32 microcontroller. Road accidents involving two-wheelers are a major concern, often caused by lack of safety measures, one of the most critical being the failure to wear helmets. Ensuring rider safety through smart systems can significantly reduce risks and improve overall road safety.

The system ensures that the rider wears a helmet before starting the vehicle by using a Force Sensitive Resistor (FSR), which detects the pressure applied when the helmet is worn. In addition, the system uses an MPU6050 sensor to detect sudden tilts or falls that may indicate an accident. In such situations, alert mechanisms such as a buzzer and LEDs are activated immediately. A manual cancel switch is also provided to prevent false alarms.

To enhance emergency response, the system integrates a GPS module to obtain real-time location coordinates. If an accident is detected and not cancelled within a defined time, the system automatically sends an alert message via WhatsApp containing the accident notification along with the precise location of the rider. This enables quick assistance and improves the chances of timely rescue.

Furthermore, the system includes an ultrasonic sensor to detect nearby obstacles and provide distance-based warnings to the rider using LEDs and a buzzer. An LDR is used for automatic headlight control, ensuring better visibility in low-light conditions.

Additionally, the project incorporates a smart direction indication system, where navigation instructions (such as left, right, or straight) are processed and displayed through LEDs, assisting the rider in following directions safely without distraction.

Overall, this project aims to develop a smart and practical safety system that enhances rider awareness, provides real-time accident detection, enables quick emergency communication through GPS and WhatsApp alerts, and improves navigation assistance, thereby making two-wheeler travel safer, smarter, and more reliable.

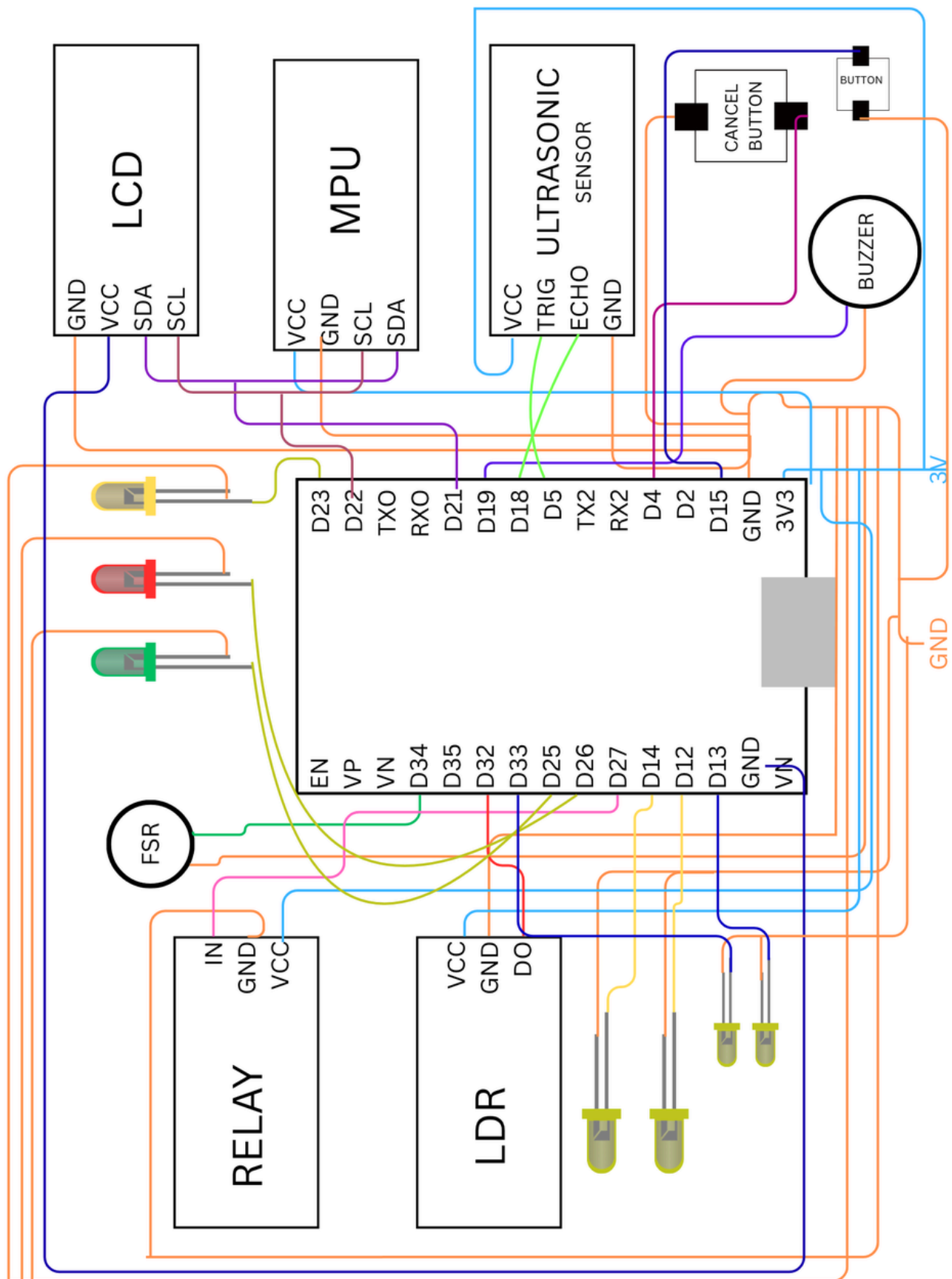
OBJECTIVE

- Ensure helmet usage before vehicle ignition using sensor-based detection
- Detect accidents using tilt and motion sensing (MPU6050)
- Provide real-time obstacle detection and proximity alerts
- Improve rider visibility through automatic headlight control using LDR
- Enable quick emergency response by sending real-time alerts via WhatsApp
- Share live accident location using GPS for faster assistance
- Provide directional assistance using visual indicators and LCD display
- Enhance overall rider safety through an integrated smart monitoring system

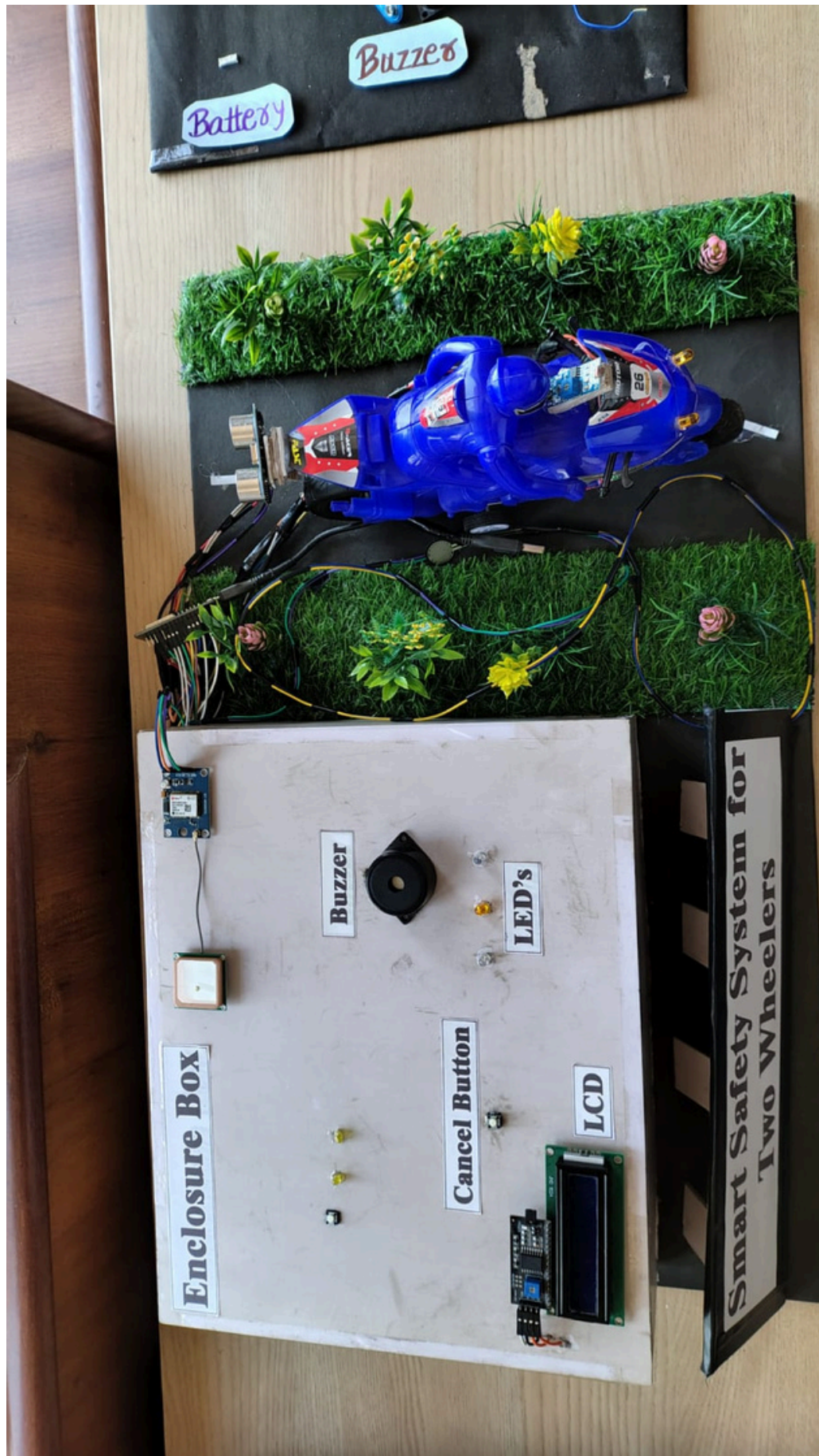
COMPONENTS USED

- ESP32 Microcontroller
- Force Sensitive Resistor (FSR)
- MPU6050 (Accelerometer + Gyroscope)
- Ultrasonic Sensor (HC-SR04)
- LDR (Light Dependent Resistor)
- Buzzer
- LEDs
- Push Button
- Neo-GPS Module
- Connecting Wires & Breadboard
- LCD Display (16x2)

CONNECTION DIAGRAM



ACTUAL PROJECT IMAGE



WORKING

The system operates by integrating multiple sensors with the ESP32 microcontroller to enhance rider safety and provide navigation assistance. Initially, the Force Sensitive Resistor (FSR) detects whether the helmet is worn. The system allows vehicle ignition only when the helmet is properly worn, ensuring basic safety compliance.

The MPU6050 sensor continuously monitors tilt and motion of the vehicle. If a sudden tilt or abnormal movement is detected, indicating a possible accident, the system activates alert mechanisms such as a buzzer and LEDs. A countdown-based alert system is implemented, allowing the rider to cancel the alert in case of a false trigger using a manual switch.

If the alert is not cancelled within the specified time, the system retrieves the rider's real-time location using the GPS module and sends an emergency notification via WhatsApp. The message includes the accident alert along with the precise latitude and longitude, enabling quick response and assistance.

The ultrasonic sensor continuously measures the distance from nearby obstacles and provides real-time warnings using LEDs and a buzzer to help prevent collisions. Additionally, an LCD display shows live distance information and directional guidance using arrows.

Directional assistance is provided through indicator LEDs and display output. Based on rider input (such as directional switches), the system indicates left or right movement using corresponding LEDs, improving communication with surrounding vehicles.

The LDR sensor enables automatic headlight control by detecting ambient light conditions. In low-light environments, the headlights are automatically activated to improve visibility and safety.

Overall, the system integrates safety enforcement, accident detection, real-time alert communication, obstacle awareness, and navigation assistance into a single smart solution, making two-wheeler travel safer and more efficient.

CODE

1.CODE FOR ESP MAIN

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <MPU6050.h>
#include <TinyGPS++.h>
#include <math.h>

// ===== PINS =====
#define FSR_PIN 34
#define RELAY 27
#define LDR_PIN 32

#define HEADLIGHT1 14
#define HEADLIGHT2 12

#define TRIG_PIN 5
#define ECHO_PIN 18

#define GREEN_LED 25
#define YELLOW_LED 26
#define RED_LED 23
#define BUZZER 19

#define CANCEL_BUTTON 4

LiquidCrystal_I2C lcd(0x27, 16, 2);
MPU6050 mpu;
// ===== GPS=====
TinyGPSPlus gps;
HardwareSerial GPSserial(2);
```

CODE

/===== VARIABLES =====

```
bool accidentMode = false;
unsigned long accidentStartTime = 0;
unsigned long lastBeepTime = 0;
```

```
void setup() {
  Serial.begin(115200);
```

```
  pinMode(FSR_PIN, INPUT);
  pinMode(RELAY, OUTPUT);
  pinMode(LDR_PIN, INPUT);
```

```
  pinMode(HEADLIGHT1, OUTPUT);
  pinMode(HEADLIGHT2, OUTPUT);
```

```
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
```

```
  pinMode(GREEN_LED, OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);
```

```
  pinMode(CANCEL_BUTTON, INPUT_PULLUP);
  digitalWrite(RELAY, HIGH);
```

```
  Wire.begin(21,22);
  lcd.init();
  lcd.backlight();
```

```
  mpu.initialize();
```


CODE

```
// GPS INIT
GPSserial.begin(9600, SERIAL_8N1, 16, 17);

lcd.setCursor(0,0);
lcd.print("SMART BIKE");
delay(2000);
lcd.clear();
}

void loop() {

// ===== GPS READ =====
while (GPSserial.available()) {
  gps.encode(GPSserial.read());
}

// ===== HELMET =====
int fsrValue = analogRead(FSR_PIN);
bool helmetWorn = (fsrValue > 500);

if (!helmetWorn) {
  digitalWrite(RELAY, HIGH);
  lcd.setCursor(0,0);
  lcd.print("Wear Helmet ");
  lcd.setCursor(0,1);
  lcd.print("Engine Locked ");
  delay(200);
  return;
}
digitalWrite(RELAY, LOW);
```

CODE

```
// ===== LDR =====  
int ldrState = digitalRead(LDR_PIN);  
if (ldrState == 1) {  
    digitalWrite(HEADLIGHT1, HIGH);  
    digitalWrite(HEADLIGHT2, HIGH);  
} else {  
    digitalWrite(HEADLIGHT1, LOW);  
    digitalWrite(HEADLIGHT2, LOW);  
}  
  
// ===== ULTRASONIC =====  
digitalWrite(TRIG_PIN, LOW);  
delayMicroseconds(5);  
  
digitalWrite(TRIG_PIN, HIGH);  
delayMicroseconds(10);  
digitalWrite(TRIG_PIN, LOW);  
  
long duration = pulseIn(ECHO_PIN, HIGH, 25000);  
float distance;  
  
if (duration == 0) distance = 999;  
else distance = (duration * 0.0343) / 2;  
if (distance < 2 || distance > 400) distance = 999;  
if (distance > 85) {  
    digitalWrite(GREEN_LED, HIGH);  
    digitalWrite(YELLOW_LED, LOW);  
    digitalWrite(RED_LED, LOW);  
    digitalWrite(BUZZER, LOW);  
}
```

CODE

```
else if (distance > 40) {  
digitalWrite(GREEN_LED, LOW);  
digitalWrite(YELLOW_LED, HIGH);  
digitalWrite(RED_LED, LOW);
```

```
digitalWrite(BUZZER, HIGH);  
delay(250);  
digitalWrite(BUZZER, LOW);  
delay(250);  
}
```

```
else {  
digitalWrite(GREEN_LED, LOW);  
digitalWrite(YELLOW_LED, LOW);  
digitalWrite(RED_LED, HIGH);
```

```
digitalWrite(BUZZER, HIGH);  
delay(120);  
digitalWrite(BUZZER, LOW);  
delay(120);  
}
```

```
/ ===== MPU =====
```

```
int16_t ax, ay, az;  
mpu.getAcceleration(&ax, &ay, &az);
```

```
float ax_g = ax / 16384.0;  
float ay_g = ay / 16384.0;  
float az_g = az / 16384.0;
```

```
float angle = atan2(  
sqrt(ax_g * ax_g + ay_g * ay_g),  
az_g  
) * 180.0 / PI;
```

CODE

```
// ===== ACCIDENT TRIGGER
```

```
=====
```

```
if (angle > 75 && !accidentMode) {  
  accidentMode = true;  
  accidentStartTime = millis();  
}
```

```
// ===== ACCIDENT MODE
```

```
=====
```

```
if (accidentMode) {
```

```
  unsigned long elapsed = millis() - accidentStartTime;  
  unsigned long remaining = 60000 - elapsed;
```

```
  int seconds = remaining / 1000;  
  int mins = seconds / 60;  
  int secs = seconds % 60;
```

```
  lcd.setCursor(0,0);  
  lcd.print("Accident Alert! ");  
  lcd.setCursor(0,1);
```

```
  if (secs < 10)  
    lcd.print(String(mins) + ":0" + String(secs) + " Cancel?");  
  else  
    lcd.print(String(mins) + ":" + String(secs) + " Cancel?");
```

```
  digitalWrite(RELAY, HIGH);  
  digitalWrite(RED_LED, HIGH);
```

CODE

```
// BUZZER
if (millis() - lastBeepTime > 500) {
  digitalWrite(BUZZER, HIGH);
  delay(100);
  digitalWrite(BUZZER, LOW);
  lastBeepTime = millis();
}

// Cancel button
if (digitalRead(CANCEL_BUTTON) == LOW) {
  accidentMode = false;
  digitalWrite(BUZZER, LOW);
  digitalWrite(RED_LED, LOW);
}

// FINAL ALERT
if (remaining <= 0) {

  lcd.setCursor(0,0);
  lcd.print("EMERGENCY !!! ");

  digitalWrite(BUZZER, HIGH);
  delay(3000);
  digitalWrite(BUZZER, LOW);

  String location;

  if (gps.location.isValid()) {
    location = String(gps.location.lat(), 6) + "," +
      String(gps.location.lng(), 6);
  } else {
    location = "0,0";
  }
}
```

CODE

```
Serial.print("ACCIDENT:");  
Serial.println(location);
```

```
accidentMode = false;  
}
```

```
return;  
}
```

```
// ===== NORMAL =====
```

```
lcd.setCursor(0,0);  
lcd.print("Dist:");  
lcd.print(distance);  
lcd.print("cm ");
```

```
lcd.setCursor(0,1);  
lcd.print("Angle:");  
lcd.print(angle);  
lcd.print(" ");
```

```
delay(150);  
}
```

CODE

2.CODE FOR WHATSAPP ALERT

```
import serial
import webbrowser
import urllib.parse
import pyautogui
import time

ser = serial.Serial('COM7', 115200)
time.sleep(2)
while True:
    data = ser.readline().decode(errors='ignore').strip()
    if data.startswith("ACCIDENT"):
        coord = data.split(":")[1]
        msg = f" Accident
detected!\nLocation:\nhttps://maps.google.com/?q={coord}"
        encoded_msg = urllib.parse.quote(msg)
        number = "91XXXXXXXXXX"
        url = f"https://web.whatsapp.com/send?phone={number}&text=
{encoded_msg}"
        print("Opening WhatsApp...")

        webbrowser.open(url)

        # wait for page load
        time.sleep(10)

        # press enter to send
        pyautogui.press("enter")

        print("Message sent!")

        break
```

CODE

3.CODE FOR DIRECTION AND INDICATOR ESP

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

#define LEFT_LED 13
#define RIGHT_LED 33
#define BUTTON 15

LiquidCrystal_I2C lcd(0x27, 16, 2);

String incoming = "";
String currentDirection = ""; //remembers last direction
bool overrideActive = false;

void setup() {
  Serial.begin(115200);

  lcd.init();
  lcd.backlight();

  pinMode(LEFT_LED, OUTPUT);
  pinMode(RIGHT_LED, OUTPUT);
  pinMode(BUTTON, INPUT_PULLUP);
  lcd.setCursor(0,0);
  lcd.print("Ready");
}

void loop() {
  // Button → turn OFF LEDs and keep them OFF
  if (digitalRead(BUTTON) == LOW) {
    overrideActive = true;
    digitalWrite(LEFT_LED, LOW);
    digitalWrite(RIGHT_LED, LOW);
```


CODE

```
delay(200); // debounce
}

// Receive serial only; don't touch LEDs here
while (Serial.available()) {
  char c = Serial.read();

  if (c == '\n') {
    processCommand(incoming);
    incoming = "";
  } else {
    incoming += c;
  }
}

void processCommand(String cmd) {
  int comma = cmd.indexOf(',');
  if (comma == -1) return;

  String direction = cmd.substring(0, comma);
  String distance = cmd.substring(comma + 1);
  // new step → reset override and update direction
  overrideActive = false;
  currentDirection = direction;
  // LCD
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print(direction);
  lcd.setCursor(0,1);
  lcd.print(distance );
}
```

CODE

```
//Update LEDs ONLY HERE (on new command)
if (!overrideActive) {
    if (currentDirection == "LEFT") {
        digitalWrite(LEFT_LED, HIGH);
        digitalWrite(RIGHT_LED, LOW);
    }
    else if (currentDirection == "RIGHT") {
        digitalWrite(LEFT_LED, LOW);
        digitalWrite(RIGHT_LED, HIGH);
    }
    else {
        digitalWrite(LEFT_LED, LOW);
        digitalWrite(RIGHT_LED, LOW);
    }
}
}
```

CODE

4.CODE FOR CALLING DIRECTIONS API

```
import requests
import serial
import time
```

```
API_KEY = "ENTER YOUR API KEY"
```

```
# SERIAL PORT
```

```
ser = serial.Serial('COM7', 115200)
```

```
time.sleep(2)
```

```
# FUNCTION
```

```
def simplify(step):
```

```
    instruction = step['html_instructions'].lower()
```

```
    distance_text = step['distance']['text']
```

```
    if "left" in instruction:
```

```
        direction = "LEFT"
```

```
    elif "right" in instruction:
```

```
        direction = "RIGHT"
```

```
    else:
```

```
        direction = "STRAIGHT"
```

```
    if "km" in distance_text:
```

```
        distance = distance_text.replace(" km", "") + "km"
```

```
    elif "m" in distance_text:
```

```
        distance = distance_text.replace(" m", "") + "m"
```

```
    else:
```

```
        distance = "0"
```

```
    return f'{direction},{distance}'
```

CODE

```
# USER INPUT
```

```
origin = input("Enter starting location: ")
destination = input("Enter destination: ")
```

```
# API CALL
```

```
url = f"https://maps.googleapis.com/maps/api/directions/json?origin={origin}&destination={destination}&key={API_KEY}"
```

```
response = requests.get(url)
data = response.json()
```

```
# ERROR CHECK
```

```
if data['status'] != "OK":
    print("Error:", data['status'])
    exit()
```

```
steps = data['routes'][0]['legs'][0]['steps']
```

```
print("\nSending directions...\n")
```

```
for step in steps:
```

```
    msg = simplify(step)
```

```
    print(msg)
```

```
    ser.write((msg + "\n").encode())
```

```
    if "km" in msg:
```

```
        value = float(msg.split(',')[1].replace("km", ""))
```

```
    elif "m" in msg:
```

CODE

```
value = float(msg.split(',')[1].replace("m", "")) / 1000
else:
    value = 0

# FINAL FIX → minimum delay added
delay = max(8, 6 + (value * 12))

print(f'Delay: {round(delay,2)} sec')

time.sleep(delay)

print("\nDone")
```

APPLICATIONS

- Two-wheeler safety systems
- Smart helmet-based safety solutions
- Accident detection and alert systems
- IoT-based vehicle safety and monitoring
- Traffic safety enhancement systems
- Smart navigation assistance systems

ADDITIONAL FEATURES

- LCD display showing real-time distance and direction
- Arrow-based navigation guidance
- Automatic left/right indication using LEDs or headlights
- Real-time accident alert via WhatsApp
- GPS-based location tracking for emergency response
- Enhanced rider communication and visibility

ADVANTAGES

- Improves overall rider safety
- Prevents vehicle ignition without helmet usage
- Enables real-time accident detection and alerting
- Provides quick emergency response through GPS and WhatsApp
- Cost-effective and scalable solution
- Efficient multi-sensor integration

FUTURE SCOPE

- Integration with GSM module for independent SMS alerts
- Dedicated mobile application for monitoring and control
- AI-based accident prediction and behavior analysis
- Integration with vehicle ignition and smart control systems
- Cloud-based data logging and analytics

CONCLUSION

The smart safety system for two-wheelers successfully demonstrates how sensor-based automation can significantly enhance rider safety. By integrating helmet detection, accident sensing, obstacle alerts, automatic lighting, GPS-based location tracking, and real-time WhatsApp alert communication, the system provides a comprehensive and intelligent safety solution. This project highlights the potential of embedded systems and IoT technologies in reducing accident risks, improving response time, and promoting safer and more responsible riding practices.